

JAX Transformer Exercises

Frank Yang

August 2026

These notes are the writeup for the coding exercises in Vlad Feinberg, *How to Land a Frontier Lab Job*. The assignment is:

1. Code a ~ 10 M transformer in JAX / Flax / Optax. Hard-code the vocab to digits 0–9, space, +, =. Train it on up-to-3-digit addition, padded to a fixed length.
2. Derive Chinchilla laws for this setup, and see how they differ for dense vs. MoE. Implement both from scratch.
3. Assuming the MoE layer used `jax.lax.ragged_dot`, write a Pallas kernel that beats ragged-dot for $F > D$ by fusing the up/down projections. Find a setting with a measurable forward-pass speedup and explain why it is there.

Relevant code can be found in the Appendix. Full notebooks can be found in this repo. They are also available on Kaggle:

- JAX transformer (dense): [yfrank792/jax-transformer](#)
- JAX transformer (MoE): [yfrank792/jax-transformer-moe](#)
- JAX transformer (dense) isoFLOP sweep: [yfrank792/jax-transformer-dense-isoflop-sweep](#)
- JAX transformer (MoE) isoFLOP sweep: [yfrank792/jax-transformer-moe-isoflop-sweep](#)
- Pallas kernel: [yfrank792/jax-transformer-moe-with-fusion-kernel](#)

Contents

1	The transformer (dense and MoE)	3
1.1	Task and data	3
1.2	Dense model	3
1.3	MoE model	3
2	Scaling laws (dense and MoE)	6
2.1	Hypothesis	6
2.2	Process (Dense)	6
2.3	Process (MoE)	7
2.4	Results	7
2.4.1	Dense	7
2.4.2	MoE	8
2.5	Analysis	8

3	Pallas kernel	12
3.1	Kernel design	12
3.2	Kernel results	12
3.3	Analysis	13
A	Relevant code	15
A.1	Fused Pallas kernel	15

1 The transformer (dense and MoE)

1.1 Task and data

Vocab $V = 13$: digits 0–9, space, +, =. Every example is a padded length- $T = 16$ string of the form

$$\mathbf{a} + \mathbf{b} = \mathbf{c}$$

with $a, b \in \{0, \dots, 999\}$ and $c = a + b$. There are exactly 10^6 such pairs; I enumerate and shuffle all of them rather than sampling with replacement.

Loss is next-token cross-entropy, masked to the answer digits only (padding and the prompt tokens do not contribute). A typical example looks like

$$\begin{aligned} x &= [5, 6, 9, 10, 11, 10, 5, 7, 5, 10, 12, 10, 1, 1, 4, 4], \\ \text{loss mask} &= [0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 1, 1, 1, 0], \end{aligned}$$

i.e. $569 + 575 = 1144$ with loss only on 1144.

1.2 Dense model

Decoder-only, pre-norm, GQA attention, RoPE, GELU MLP, residual stream. No gated MLP (just $W_{\text{in}}, W_{\text{out}}$). Pilot config:

config	value
<code>n_layers</code> (L)	12
<code>d_model</code> (D)	256
<code>d_ff</code> (F)	1024
<code>n_heads</code> (N)	16
<code>n_kv_heads</code> (K)	2
<code>d_qkv</code> (H)	16
<code>n_embed</code> (V)	13
params	$\approx 8.07 \times 10^6$

Parameter count, matching the scaling-book notation:

$$\#\text{params} = 2D(N + K)HL + 2DFL + 2DV.$$

Trained on an 8-chip TPU mesh with pure data parallelism over the batch axis. Batch size 4000 sequences (64000 tokens), chosen so the per-chip batch sits above the ICI-bound threshold $B > CX/W_{\text{ICI}} \approx 35000$ tokens from the scaling book. AdamW + cosine decay, 20 epochs, 80/20 train/eval split of the 10^6 pairs.

The $\sim 8\text{M}$ model solves the task. Eval loss is 0.57 after epoch 0 and $\approx 8.5 \times 10^{-4}$ by epoch 19. The loss curve can be found at Figure 1.

1.3 MoE model

Same data and attention stack as the dense transformer. The MLP is replaced by a top- k routed expert FFN with $E = 8$ experts, $k = 2$ active. Routing is a linear map $W_r \in \mathbb{R}^{D \times E}$, softmax, top- k , then renormalize the selected gates. Tokens are sorted by expert id and dispatched with `lax.ragged_dot` (custom VJP for backward pass).

Pilot config:

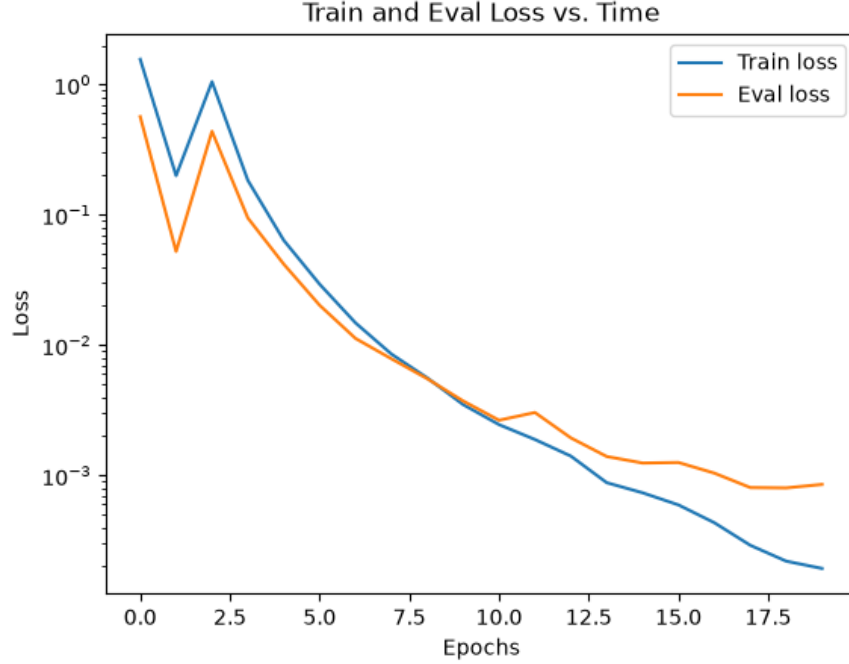


Figure 1: Dense 8M pilot: train and eval loss vs. epoch (y -axis log-scaled).

config	value
<code>n_layers</code> (L)	12
<code>d_model</code> (D)	256
<code>d_ff</code> (F)	512
<code>n_heads</code> (N)	16
<code>n_kv_heads</code> (K)	2
<code>d_qkv</code> (H)	16
<code>n_experts</code> (E)	8
<code>n_active</code> (k)	2
<code>n_embed</code> (V)	13
active params	$\approx 8.09 \times 10^6$
total params	$\approx 2.70 \times 10^7$

$$\begin{aligned} \text{total params} &= 2D(N + K)HL + 2DEFL + 2DV + DEL \approx 2.70 \times 10^7, \\ \text{active params} &= 2D(N + K)HL + 2DkFL + 2DV + DEL \approx 8.09 \times 10^6. \end{aligned}$$

The MoE is an 8M-active / 27M-total model at the same active FLOPs-per-token as the dense 8M.

It also solves the task: eval loss 0.50 after epoch 0, $\approx 1.2 \times 10^{-3}$ by epoch 19. The loss curve can be found at Figure 2.

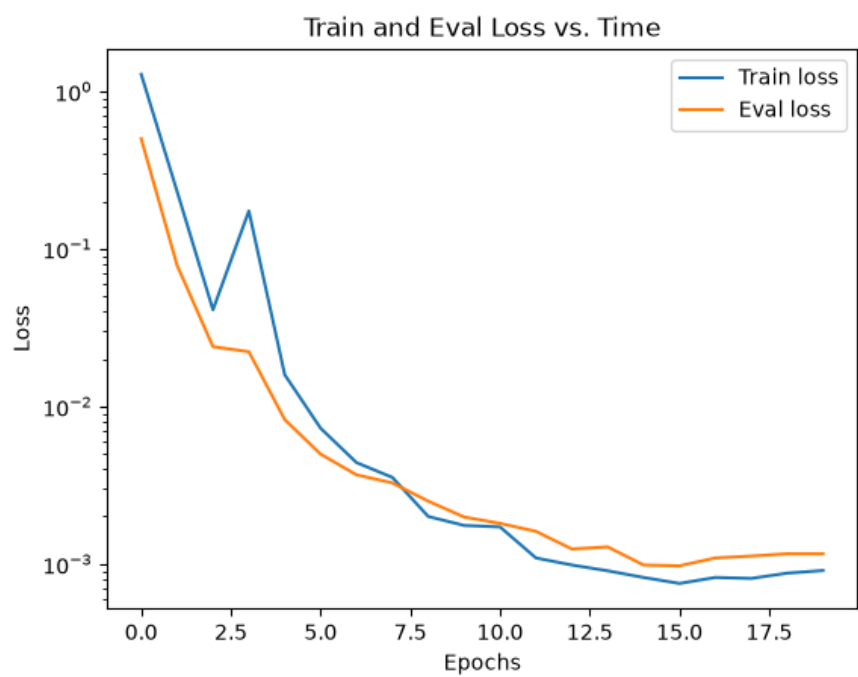


Figure 2: MoE pilot: train and eval loss vs. epoch (y -axis log-scaled).

2 Scaling laws (dense and MoE)

2.1 Hypothesis

Hoffmann et al. [1] found that for compute-optimal training for LLMs, the model size and number of tokens should be scaled equally as FLOPs increased. I hypothesized a similar relationship for transformers being trained on 3-digit addition. To test this hypothesis, I implemented the second approach (IsoFLOP profiles) Hoffmann et al. utilized: for a fixed set of FLOP budgets C , vary model sizes N , and train each on the corresponding number of tokens D , calculated as

$$D \approx \frac{C}{6N}$$

and read off $N^*(C)$ as the N that minimizes eval loss on that slice. Then, I attempted to fit a power law relationship as Hoffmann et al. did:

$$N^*(C) \propto C^a, \quad D^*(C) \propto C^b,$$

hypothesizing that I would reach similar values for $a \approx b \approx 1/2$ (equal split of extra compute into params and tokens).

2.2 Process (Dense)

To stay as close as possible to a unique-token Chinchilla setup I restricted every run to ≤ 1 epoch. FLOP budgets

$$C \in \{5.0, 5.95, 7.07, 8.41, 10\} \times 10^{11}$$

(a geometric grid from 5×10^{11} to 10^{12}). For each C I only include models with $D(C, N) \leq$ one epoch, then LR-sweep and pick the best of 3 seeds. I use a train/eval split of 90/10.

Dense family ($F = 4D$):

d_model (D)	n_layers (L)	n_heads (N)	n_kv_heads (K)	d_qkv (H)	d_ff (F)	n_embed (V)	params N
12	2	2	2	16	48	13	5.8×10^3
16	2	2	2	16	64	13	8.8×10^3
20	2	2	2	16	80	13	1.2×10^4
24	2	2	2	16	96	13	1.6×10^4
32	2	2	2	16	128	13	2.6×10^4
48	2	2	2	16	192	13	5.1×10^4
64	3	4	2	16	256	13	1.4×10^5
80	4	4	2	16	320	13	2.7×10^5
96	5	6	2	16	384	13	5.0×10^5
128	8	8	2	16	512	13	1.4×10^6
160	10	10	2	16	640	13	2.7×10^6
256	12	16	2	16	1024	13	8.1×10^6

While I include models with widths 80 and above in the training family and my initial experiments, ultimately, for the fixed set of FLOPs budgets, they underperform the smaller models significantly, and are thus excluded from the reported slices.

FLOPs per run are counted as $C = 6ND$ with N = total dense params. Each (C, N) point is an LR grid $\{10^{-4}, 2 \times 10^{-4}, \dots, 8 \times 10^{-3}\} \times$ seeds $\{42, 43, 44\}$; I keep the best mean eval loss and record its standard deviation across 3 seeds.

2.3 Process (MoE)

Same ≤ 1 epoch constraint as the dense sweep, so no repeated tokens. $E = 8$, $k = 2$, $F = 2D$ so active FFN FLOPs match the dense $F = 4D$ family. Active params and FLOPs are

$$N_{\text{active}} = N_{\text{total}} - N_{\text{expert}} \left(1 - \frac{k}{E}\right), \quad C = 6 N_{\text{active}} D.$$

Each C_j is one epoch of model j , for the five smallest widths:

$$C_j = 6 N_{\text{active},j} D_{\text{epoch}}, \quad D_{\text{epoch}} = 0.9 \times 10^6 \times 16 = 1.44 \times 10^7,$$

$$C \in \{5.18, 7.80, 10.9, 14.3, 22.7\} \times 10^{11}.$$

The slice at C_j keeps only models with $N \geq N_j$ (so $n_{\text{epochs}} \leq 1$): range $[j, 10]$, left edge = model j . I use a train/eval split of 90/10.

MoE family ($F = 2D$):

d_model (D)	n_layers (L)	n_heads (N)	n_kv_heads (K)	d_qkv (H)	d_ff (F)	n_experts (E)	n_active (k)	n_embed (V)	active N_{active}	total N_{total}
12	2	2	2	16	24	8	2	13	6.0×10^3	1.3×10^4
16	2	2	2	16	32	8	2	13	9.0×10^3	2.1×10^4
20	2	2	2	16	40	8	2	13	1.3×10^4	3.2×10^4
24	2	2	2	16	48	8	2	13	1.7×10^4	4.4×10^4
32	2	2	2	16	64	8	2	13	2.6×10^4	7.5×10^4
48	2	2	2	16	96	8	2	13	5.2×10^4	1.6×10^5
64	3	4	2	16	128	8	2	13	1.4×10^5	4.3×10^5
80	4	4	2	16	160	8	2	13	2.7×10^5	8.9×10^5
96	5	6	2	16	192	8	2	13	5.0×10^5	1.6×10^6
128	8	8	2	16	256	8	2	13	1.4×10^6	4.5×10^6
160	10	10	2	16	320	8	2	13	2.7×10^6	8.8×10^6

Unlike with the dense transformer sweep, widths $D \geq 80$ are included in the reported slices. Similar to the dense transformer sweep, they ultimately underperform compared with their smaller counterparts.

Each (C, N) point uses the same LR grid $\{10^{-4}, 2 \times 10^{-4}, \dots, 8 \times 10^{-3}\} \times \text{seeds } \{42, 43, 44\}$; I keep the best mean eval loss and record its standard deviation across 3 seeds.

2.4 Results

Both sweeps follow similar patterns. A few slices display a shallow, unsteady U-curve; the rest rise from the left edge. In every case the valley, if it exists, stays at a few thousand parameters and does not increase to larger N as C grows. The right side, with greater numbers of parameters, consistently underperforms.

2.4.1 Dense

Eval loss at the best LR $\times$ seed, geometric C grid. Bold is N^* on that slice. Larger widths are omitted.

D	N	$C=5.00\text{e}11$	$5.95\text{e}11$	$7.07\text{e}11$	$8.41\text{e}11$	$1.00\text{e}12$
12	5.8k	0.213	—	—	—	—
16	8.8k	0.123	0.157	0.113	—	—
20	12k	0.130	0.134	0.118	0.096	0.117
24	16k	0.155	0.143	0.135	0.107	0.108
32	26k	0.174	0.138	0.152	0.140	0.114
48	51k	0.520	0.281	0.391	0.273	0.292
64	0.14M	1.27	1.11	1.05	0.407	0.619

$N^*(C)$ is

8.8k, 12k, 8.8k, 12k, 16k.

A log-log fit gives $a \approx 0.71$ ($R^2 = 0.61$), $b \approx 0.29$ ($R^2 = 0.13$). The first slice is a real (shallow) U: $0.213 \rightarrow 0.123 \rightarrow 0.130$ and then up. $C = 5.95 \times 10^{11}$ and $C = 10^{12}$ dip once and rise; the other two are monotone from the left edge. The argmin hops among the the two smallest widths available in each FLOPs budget, and three smallest widths of the family. The isoFLOP curves and the N^* , D^* power-law fit can be found at Figure 3.

2.4.2 MoE

Eval loss at the best LR $\times$ seed, geometric C grid. Bold is N^* on that slice in active params.

D	L	N_{active}	N_{total}	$C=5.18\text{e}11$	$7.80\text{e}11$	$1.09\text{e}12$	$1.43\text{e}12$	$2.27\text{e}12$
12	2	6.0k	13k	0.087	—	—	—	—
16	2	9.0k	21k	0.107	0.089	—	—	—
20	2	12.6k	32k	0.141	0.062	0.080	—	—
24	2	16.6k	44k	0.111	0.096	0.083	0.087	—
32	2	26k	75k	0.188	0.143	0.132	0.060	0.034
48	2	52k	0.16M	0.753	0.457	0.200	0.234	0.092
64	3	0.14M	0.43M	1.09	0.247	0.195	0.142	0.094
80	4	0.27M	0.89M	1.43	1.20	0.536	0.345	0.130
96	5	0.50M	1.6M	1.58	1.38	1.19	1.04	0.356
128	8	1.4M	4.5M	1.93	1.80	1.63	1.56	1.37
160	10	2.7M	8.8M	2.08	2.01	1.91	1.81	1.63

$N^*(C)$ in active params is

6.0k, 12.6k, 12.6k, 26k, 26k.

A log-log fit gives $a \approx 1.0$ ($R^2 = 0.76$), $b \approx -0.02$ ($R^2 \approx 0$). The argmin is the left edge of the slice on 3 of 5 budgets, and one step to the right on the other two. Everything further right depicts subpar performance. The isoFLOP curves and the N^* , D^* power-law fit can be found at Figure 4.

2.5 Analysis

I hypothesize that neither sweep derives a Chinchilla law because the task saturates.

Hoffmann et al. [1] fit the parametric loss

$$L(N, D) = E + \frac{A}{N^\alpha} + \frac{B}{D^\beta},$$

where E is the irreducible entropy of the data. Language has $E > 0$: an infinite model on infinite tokens still has leftover uncertainty.

3-digit addition, by comparison, is a small, deterministic task, that can be implemented with a simple deterministic algorithm. There is no entropy floor. The A/N^α term also collapses as soon as N is large enough to learn digit-wise addition and digit carrying, and that threshold is a few thousand parameters. The data is further constrained: one epoch is 0.9×10^6 pairs, $\approx 1.44 \times 10^7$ tokens, after which we are repeating the same data.

IsoFLOP sweeps I conducted therefore measure how fast a given width reaches the task floor, not a Hoffmann compute-optimal allocation. Constraining to one epoch is necessary but not sufficient; the bottleneck is task capacity, not unique tokens. Thus, fitted power-law exponents are unreliable, and dense vs. MoE Chinchilla exponents are not identified on this task.

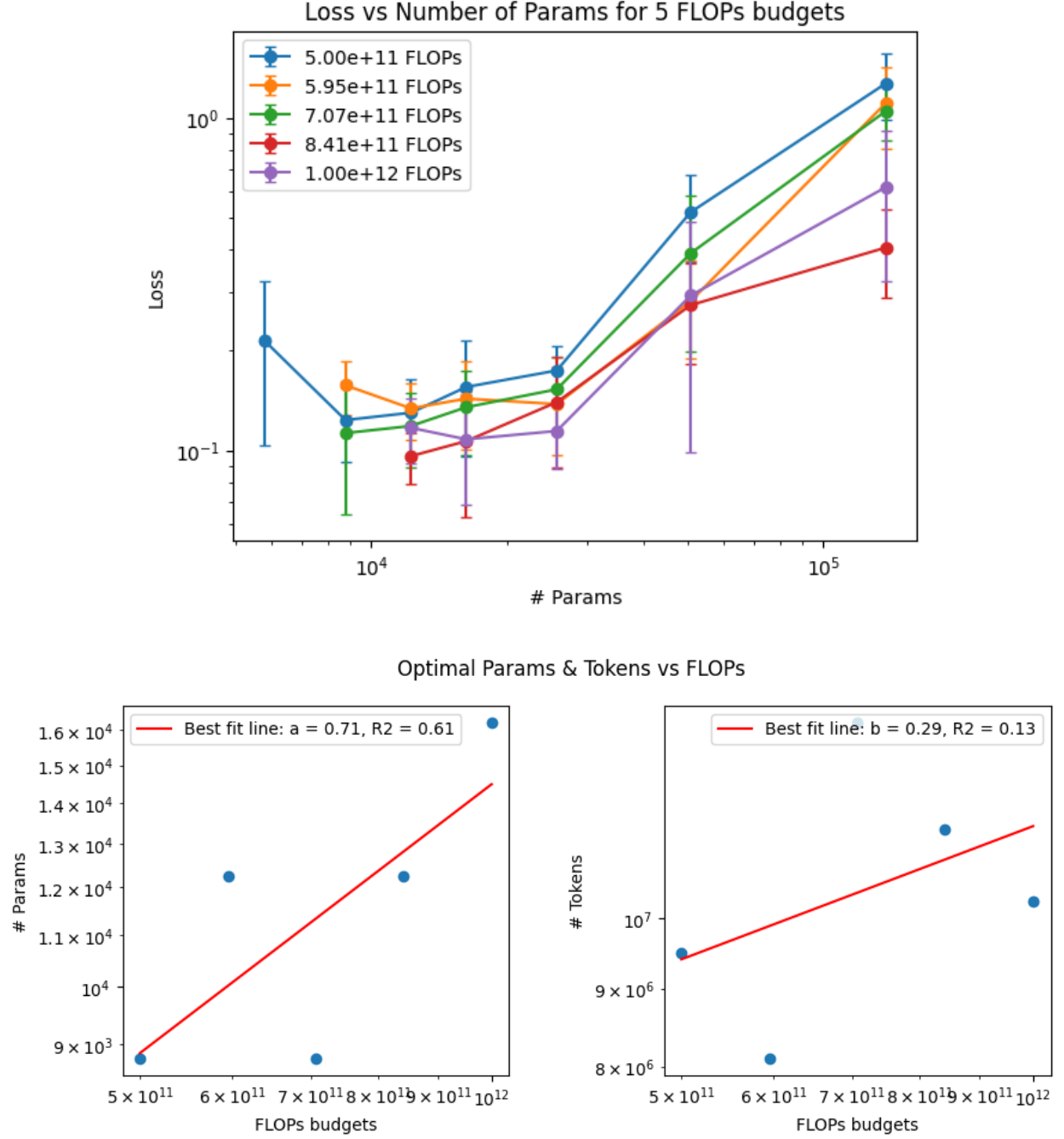


Figure 3: Dense isoFLOP sweep. Top: eval loss vs. N at five compute budgets (error bars are seed std). Bottom: compute-optimal $N^*(C)$ and $D^*(C)$, with fitted exponents $a \approx 0.71$ ($R^2 = 0.61$) and $b \approx 0.29$ ($R^2 = 0.13$).

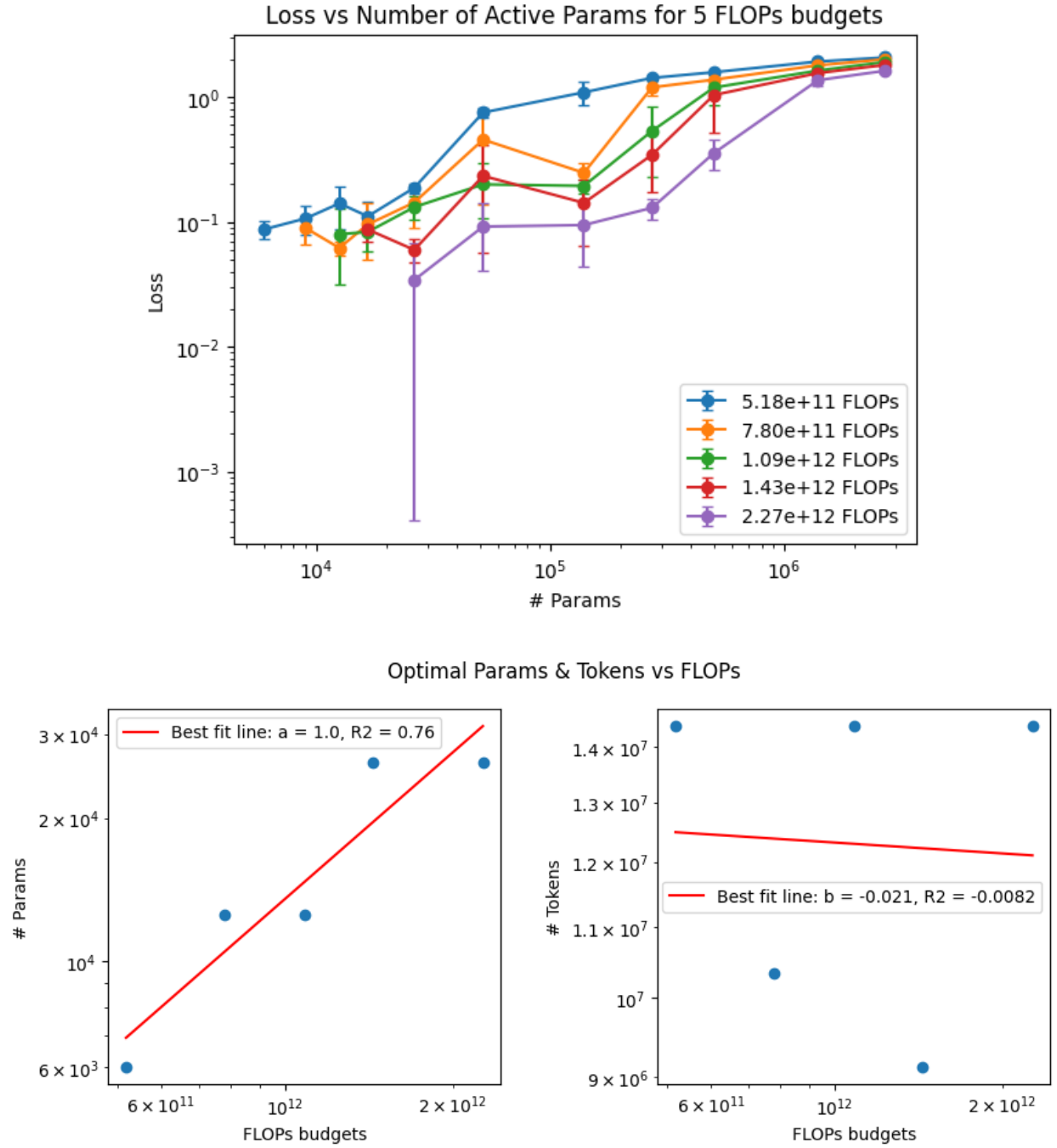


Figure 4: MoE isoFLOP sweep, in active params. Top: eval loss vs. N_{active} at five compute budgets (error bars are seed std). Bottom: compute-optimal $N^*(C)$ and $D^*(C)$, with fitted exponents $a \approx 1.0$ ($R^2 = 0.76$) and $b \approx -0.02$ ($R^2 \approx 0$).

3 Pallas kernel

3.1 Kernel design

Let B_{tok} denote the number of dispatched token copies on one accelerator. After routing and sorting, the MoE FFN is two ragged grouped matmuls with a GELU in between:

$$y = \text{GELU}(x W_{\text{in}}^{(e)}) W_{\text{out}}^{(e)},$$

$x \in \mathbb{R}^{B_{\text{tok}} \times D}$, $W_{\text{in}}^{(e)} \in \mathbb{R}^{D \times F}$, $W_{\text{out}}^{(e)} \in \mathbb{R}^{F \times D}$, one expert e per token-group.

Unfused `lax.ragged_dot` materializes the $B_{\text{tok}} \times F$ activation in HBM:

$$x \xrightarrow{\text{HBM}} \underbrace{x W_{\text{in}}}_{\in \mathbb{R}^{B_{\text{tok}} \times F}} \xrightarrow{\text{HBM}} (\cdot) W_{\text{out}} \xrightarrow{\text{HBM}} y.$$

I fuse the two ragged group matmuls and the activation, so the $B_{\text{tok}} \times F$ tensor is never written to HBM. For each F -slice, the first dot produces a $(256, \text{blk_F})$ GELU intermediate that remains local to the program. The second dot contracts it to a $(256, D)$ partial output.

I implemented this as a MegaBlocks-style grouped GEMM: preprocess the sorted tokens into tiles of size `block_size` = 256, with a masked tile where an expert boundary interrupts a full tile; Pallas grid (`#tiles`, $F/\text{blk_F}$), with `#tiles` = $B_{\text{tok}}/256 + E - 1$ in this benchmark; each program: load a $(256, D)$ activation tile and a $(D, \text{blk_F}) / (\text{blk_F}, D)$ weight slice; compute $\text{GELU}(x W_{\text{in}}) W_{\text{out}}$ for that F -slice; then mask and read-modify-write the partial result into the output tile.

The code is attached in the Appendix.

3.2 Kernel results

I benchmarked the MoE MLP functions on a TPU v5e-8 stack with pure data sharding. Activations are bf16; weights are stored in fp32 and cast to bf16 before the matmuls. The baseline is the original MLP implemented with `lax.ragged_dot`. Fixed config:

config	value
<code>d_model</code> (D)	2048
<code>n_experts</code> (E)	8
<code>n_active</code> (k)	2
sequence batch B_{seq}	2048
sequence length T	16
<code>block_size</code>	256

I sweep F/D at fixed $D = 2048$:

$$F/D \in \{1/8, 1/4, 1/2, 1, 2, 4, 8\}, \quad F \in \{256, 512, 1024, 2048, 4096, 8192, 16384\}.$$

Each (D, F) point uses 5 seeds, each with regenerated weights and inputs, a check against `ragged_dot` (atol/rtol 10^{-2}), then a timed run. A run is compiled once with `block_until_ready`, then `timeit` over 100 trials; I report the mean and standard deviation over seeds for each point.

Token tiles are a fixed `block_size` = 256 (a multiple of 8). `blk_F` is the approximate, largest power-of-two split of F whose two weight slices $W_{\text{in}} \in \mathbb{R}^{D \times \text{blk_F}}$ and $W_{\text{out}} \in \mathbb{R}^{\text{blk_F} \times D}$ fit in a 2×10^6 -element VMEM budget. For every $F \geq 512$ in this sweep `blk_F` = 512.

F	F/D	blk_F	ragged_dot (s)		fused (s)	winner, ratio
256	1/8	256	$1.368\text{e-}3 \pm 3.09\text{e-}5$	$1.515\text{e-}3 \pm 8.89\text{e-}6$		ragged, 0.90×
512	1/4	512	$1.507\text{e-}3 \pm 4.68\text{e-}5$	$2.025\text{e-}3 \pm 1.33\text{e-}5$		ragged, 0.74×
1024	1/2	512	$2.093\text{e-}3 \pm 8.10\text{e-}5$	$2.419\text{e-}3 \pm 2.26\text{e-}5$		ragged, 0.87×
2048	1	512	$2.823\text{e-}3 \pm 1.73\text{e-}5$	$3.318\text{e-}3 \pm 7.04\text{e-}5$		ragged, 0.85×
4096	2	512	$4.944\text{e-}3 \pm 5.68\text{e-}5$	$4.909\text{e-}3 \pm 1.02\text{e-}4$		tie, 1.01×
8192	4	512	$9.791\text{e-}3 \pm 4.16\text{e-}5$	$8.134\text{e-}3 \pm 4.47\text{e-}5$		fused, 1.20×
16384	8	512	$2.692\text{e-}2 \pm 3.72\text{e-}5$	$1.448\text{e-}2 \pm 2.51\text{e-}5$		fused, 1.86×

The transition occurs near $F = 2D$, where the difference is smaller than the seed-to-seed variation and is therefore treated as a tie. **ragged_dot** wins for $F \leq D$; the fused kernel has a clear advantage from $F = 4D$, reaching 1.86× at $F = 8D$. The settings with clear measured speedups are $D = 2048, F = 8192$ and $D = 2048, F = 16384$.

3.3 Analysis

Fusion removes the write and subsequent read of the $B_{\text{tok}} \times F$ intermediate, saving $4B_{\text{tok}}F$ bytes for a bf16 activation. We can estimate the savings by counting the number of saved bytes vs. total bytes communicated from and to HBM.

	unfused	fused
input x	$2B_{\text{tok}}D$	$2B_{\text{tok}}D$
weights W_{in}	$4EDF$	$4EDF$
intermediate act	$4B_{\text{tok}}F$	0
weights W_{out}	$4EDF$	$4EDF$
output	$2B_{\text{tok}}D$	$2B_{\text{tok}}D$

The saving is linear in F and becomes a larger fraction of compulsory traffic as F/D grows:

$$\frac{\text{bytes saved}}{\text{unfused bytes}} = \frac{4B_{\text{tok}}F}{8EDF + 4B_{\text{tok}}D + 4B_{\text{tok}}F} = \frac{B_{\text{tok}}}{2ED + B_{\text{tok}} + B_{\text{tok}}D/F}.$$

Per-chip tokens after top- k dispatch are $B_{\text{tok}} = B_{\text{seq}}Tk/8 = 2048 \times 16 \times 2/8 = 8192$. With $D = 2048$ and $E = 8$, the idealized ratio is $\approx 7.7\%$ at $F = D/8$ and $\approx 19.5\%$ at $F = 8D$.

The fused grid also incurs preprocessing and masking costs. Those costs dominate for small F , where the fused implementation loses. At large F , avoiding materializing intermediate activations in HBM between two grouped GEMMs outweighs this overhead.

At $F = 8D$, eliminating 19.5% of the idealized compulsory traffic would produce only a $1/(1 - 0.195) \approx 1.24\times$ speedup if runtime were proportional to those bytes, below the measured 1.86×

I hypothesize that the remaining gain comes from execution effects omitted by pure bytes counting, such as additional memory movement and less efficient hardware utilization by **ragged_dot**. Supporting this interpretation, doubling F from $4D$ to $8D$ doubles the MLP arithmetic but increases **ragged_dot** latency by 2.75×, compared with 1.78× for the fused kernel.

References

- [1] Jordan Hoffmann, Sebastian Borgeaud, Arthur Mensch, et al. Training compute-optimal large language models. arXiv:2203.15556, 2022.

A Relevant code

A.1 Fused Pallas kernel

```
# megablocks GMM approach

def MoE_proj(x, W_in, W_out, idx, blk_F, block_size=256):
    B, D = x.shape
    E, _, F = W_in.shape

    # block_size = 256 # multiple of 8
    n_blocks = B // block_size
    n_tiles = n_blocks + E - 1

    # preprocessing

    experts = jnp.zeros((n_tiles,), dtype=jnp.int32)
    indices = jnp.zeros((n_tiles,), dtype=jnp.int32)
    masks = jnp.ones((n_tiles, block_size), dtype=jnp.int32)

    idx = jnp.pad(idx, (0, block_size))
    x = jnp.pad(x, ((0, block_size), (0, 0)))

    def fun(i, carries):
        j, cur_expert, experts, indices, masks = carries

        cur_idx = lax.dynamic_slice(idx, (j,), (block_size,))

        indices = indices.at[i].set(j)
        experts = experts.at[i].set(jnp.minimum(cur_expert, E-1))
        masks = lax.dynamic_update_slice(masks,
                                         jnp.where(cur_idx == cur_expert, 1, 0).reshape(-1,
                                                block_size),
                                         (i, 0))

        j, new_expert = lax.cond(
            idx[j+block_size-1] == cur_expert,
            lambda j,e: (j+block_size, e),
            lambda j,e: (j, e+1),
            j, cur_expert
        )
        new_expert = lax.cond(
            idx[j] > new_expert,
            lambda e: idx[j],
            lambda e: e,
            new_expert
        )

    return j, new_expert, experts, indices, masks

_, _, experts, indices, masks = lax.fori_loop( # indices = element indices
    0,
    n_tiles,
    fun,
    (0, 0, experts, indices, masks)
)
```

```

# grid -> 2 loops
# outer loop over tiles, inner loop over F

def MoE_kernel(
    experts, indices, # prefetched scalars
    x_ref, W_in_ref, W_out_ref, mask_ref, # input refs,
    o_ref, # output ref
    accum_scratch, # scratch buffer
):
    del experts

    is_start = pl.program_id(1) == 0
    first_block = pl.program_id(0) == 0
    new_block = indices[jnp.maximum(0, pl.program_id(0)-1)] != indices[pl.program_id(0)]

    @pl.when(is_start & (first_block | new_block))
    def _():
        o_ref[...] = jnp.zeros_like(o_ref)

    accum_scratch[...] = lax.dot(
        lhs=jax.nn.gelu(
            lax.dot(
                lhs=x_ref [...],
                rhs=W_in_ref[0, :, :],
                preferred_element_type=jnp.float32,
            )
        ).astype(x_ref.dtype),
        rhs=W_out_ref[0, :, :],
        preferred_element_type=jnp.float32,
    )

    o_ref[...] += (accum_scratch[...] * mask_ref[0, :, :]).astype(o_ref.dtype)

def x_map(i, j, experts, indices):
    del j, experts
    return (indices[i] // block_size, 0)

def W_in_map(i, j, experts, indices):
    del indices
    return (experts[i], 0, j)

def W_out_map(i, j, experts, indices):
    del indices
    return (experts[i], j, 0)

def mask_map(i, j, experts, indices):
    del j, experts, indices
    return (i, 0, 0)

def o_map(i, j, experts, indices):
    del j, experts
    return (indices[i] // block_size, 0)

masks = lax.broadcast_in_dim(masks, (*masks.shape, D), broadcast_dimensions=(0,1))

grid_spec = pltpu.PrefetchScalarGridSpec(

```

```

        num_scalar_prefetch=2,
        grid=(n_tiles, F // blk_F),
        in_specs=[
            pl.BlockSpec((block_size, D), index_map=x_map),
            pl.BlockSpec((1, D, blk_F), index_map=W_in_map),
            pl.BlockSpec((1, blk_F, D), index_map=W_out_map),
            pl.BlockSpec((1, block_size, D), index_map=mask_map)
        ],
        out_specs=pl.BlockSpec((block_size, D), index_map=o_map),
        scratch_shapes=[
            pltpu.VMEM((block_size, D), dtype=jnp.float32)
        ]
    )

    kernel = pl.pallas_call(
        MoE_kernel,
        grid_spec=grid_spec,
        out_shape=jax.ShapeDtypeStruct.like(x)
    )

    out = kernel(experts, indices, x, W_in, W_out, masks)

    out = out.at[: -block_size].get()

    return out

@jax.jit(
    static_argnames=["k", "blk_F", "block_size"]
)
def mlp_with_kernel(x: jnp.array, W_r: jnp.array, k: jnp.int32, W_in: jnp.array, W_out: jnp.
    array, blk_F: jnp.int32, block_size: jnp.int32) -> jnp.array:

    B = x.shape[0]
    T = x.shape[1]
    D = x.shape[2]
    E = W_in.shape[0]
    F = W_in.shape[2]

    @jax.shard_map(
        in_specs=(P('X'), P('X'), None), P(None, None, None), P(None, None, None)),
        out_specs=P('X'), None),
        check_vma=False
    )
    def local_ragged_dot(idx, x, W_in, W_out):
        perm = jnp.argsort(idx)
        x_sorted = x[perm]
        idx_sorted = idx[perm]

        out_sorted = MoE_proj(x_sorted, W_in, W_out, idx_sorted, blk_F, block_size)

        perm_inv = jnp.argsort(perm)

        out = out_sorted[perm_inv]

    return out

W_r = W_r.astype(x.dtype) # consider if needed later
W_in = W_in.astype(x.dtype)

```

```

W_out = W_out.astype(x.dtype)

routing = jnp.einsum('btd,de->bte', x, W_r)
routing = jax.nn.softmax(routing) # in case we end up with 0

gate, idx = lax.top_k(routing, k)

gate = gate / gate.sum(axis=-1, keepdims=True, dtype=jnp.float32) # temporarily set as
float32 for greater precision
gate = gate.astype(x.dtype)

idx_flat = idx.flatten()

x_repeat = jnp.repeat(x, k, axis=1)
x_repeat = x_repeat.reshape((-1, D))

out_repeat = local_ragged_dot(idx_flat, x_repeat, W_in, W_out)
out_repeat = out_repeat.reshape((B, T, -1, D))

out = jnp.einsum('btkd,btk->btd', out_repeat, gate)

return out

```